

---

# SPARK

Sistema de Busca de Informações de Pesquisadores

---

## Plano de Execução do Pipeline ETL

*Sprint II — SPK-73 — Camada Operacional*

**Equipe:** Calvin Albuquerque · Glenda Santana

**Versão:** 1.1

**Data:** May 19, 2026

**Projeto:** UNEB — Tópicos Especiais em Engenharia de Software

---

## Contents

---

|           |                                                     |           |
|-----------|-----------------------------------------------------|-----------|
| <b>1</b>  | <b>Visão Geral do Pipeline</b>                      | <b>2</b>  |
| <b>2</b>  | <b>Fontes de Entrada</b>                            | <b>2</b>  |
| 2.1       | XMLs dos Currículos Lattes . . . . .                | 2         |
| 2.2       | Planilha Qualis CAPES . . . . .                     | 3         |
| 2.3       | CrossRef API . . . . .                              | 4         |
| 2.4       | OpenAlex API . . . . .                              | 5         |
| <b>3</b>  | <b>Fase 1 — Extração dos XMLs</b>                   | <b>5</b>  |
| 3.1       | Configuração no Apache Hop . . . . .                | 5         |
| 3.2       | Mapeamento por Tipo de Produção . . . . .           | 6         |
| <b>4</b>  | <b>Fase 2 — Transformação e Normalização</b>        | <b>6</b>  |
| 4.1       | Limpeza de ISSN . . . . .                           | 6         |
| 4.2       | Normalização de Títulos . . . . .                   | 6         |
| 4.3       | Deduplicação e UPSERT . . . . .                     | 7         |
| <b>5</b>  | <b>Fase 3 — Enriquecimento</b>                      | <b>7</b>  |
| 5.1       | Lookup Qualis (CSV local) . . . . .                 | 7         |
| 5.2       | Lookup DOI (CrossRef API) . . . . .                 | 8         |
| 5.3       | Lookup Impact Factor (OpenAlex API) . . . . .       | 8         |
| <b>6</b>  | <b>Fase 4 — Carga no Supabase</b>                   | <b>9</b>  |
| 6.1       | Conexão JDBC . . . . .                              | 9         |
| 6.2       | Ordem de Carga . . . . .                            | 9         |
| 6.3       | UPSERT de Pesquisadores . . . . .                   | 9         |
| 6.4       | UPSERT de Produções . . . . .                       | 9         |
| 6.5       | Registro de Log . . . . .                           | 10        |
| <b>7</b>  | <b>Fase 5 — Acionamento do Worker de Embeddings</b> | <b>10</b> |
| <b>8</b>  | <b>Mapa Completo de Correlação de Dados</b>         | <b>11</b> |
| <b>9</b>  | <b>Tratamento de Erros e Logs</b>                   | <b>11</b> |
| 9.1       | Erros por arquivo XML . . . . .                     | 11        |
| 9.2       | Erros de API . . . . .                              | 11        |
| 9.3       | Campos sem correspondência . . . . .                | 11        |
| <b>10</b> | <b>Checklist de Pré-Execução</b>                    | <b>12</b> |
| <b>11</b> | <b>Referências</b>                                  | <b>12</b> |

## 1. Visão Geral do Pipeline

---

O pipeline ETL do SPARK é responsável por ler os arquivos XML dos currículos Lattes, extrair e normalizar os dados, enriquecer cada produção com informações de fontes externas e carregar tudo no banco Supabase. A execução é orquestrada pelo Apache Hop e ocorre uma vez por ciclo de atualização de currículos.

O pipeline executa cinco fases em sequência:

1. **Extração** — leitura e parse dos XMLs do Lattes
2. **Transformação** — normalização, limpeza e mapeamento para o schema do banco
3. **Enriquecimento** — lookup de Qualis (CSV), DOI (CrossRef) e Impact Factor (OpenAlex)
4. **Carga** — UPSERT no Supabase
5. **Embeddings** — acionamento do worker de geração de vetores

***Nota:** um pesquisador sem produções bibliográficas (currículo enxuto ou recém-cadastrado) ainda é inserido na tabela `pesquisadores`. O pipeline não falha por ausência de produções — apenas registra zero itens no log.*

## 2. Fontes de Entrada

---

### 2.1 XMLs dos Currículos Lattes

Os arquivos XML são obtidos manualmente pelo Engenheiro de Dados via download no portal Lattes ([lattes.cnpq.br](https://lattes.cnpq.br)). Cada arquivo segue o padrão oficial do CNPq com encoding ISO-8859-1 e tag raiz `CURRICULO-VITAE`.

Campos relevantes para extração:

| Nó XML                      | Atributo / Tag         | Campo no banco                           |
|-----------------------------|------------------------|------------------------------------------|
| CURRICULO-VITAE             | NUMERO-IDENTIFICADOR   | pesquisadores.lattes_id                  |
| DADOS-GERAIS                | NOME-COMPLETO          | pesquisadores.nome_completo              |
| DADOS-GERAIS                | ORCID-ID               | informativo (não armazenado)             |
| RESUMO-CV                   | TEXTO-RESUMO-CV-RH     | pesquisadores.resumo                     |
| ARTIGO ><br>DADOS-BASICOS   | TITULO-DO-ARTIGO       | producoes.titulo                         |
| ARTIGO ><br>DADOS-BASICOS   | ANO-DO-ARTIGO          | producoes.ano_publicacao                 |
| ARTIGO ><br>DADOS-BASICOS   | DOI                    | producoes.doi (se preenchido)            |
| ARTIGO ><br>DETALHAMENTO    | TITULO-DO-PERIODICO-OU | producoes.nome_veiculo                   |
| ARTIGO ><br>DETALHAMENTO    | ISSN                   | producoes.issn (chave de enriquecimento) |
| EVENTO ><br>DADOS-BASICOS   | TITULO-DO-TRABALHO     | producoes.titulo<br>(tipo=EVENTO)        |
| EVENTO ><br>DADOS-BASICOS   | ANO-DO-TRABALHO        | producoes.ano_publicacao                 |
| LIVRO ><br>DADOS-BASICOS    | TITULO-DO-LIVRO        | producoes.titulo<br>(tipo=LIVRO)         |
| CAPITULO ><br>DADOS-BASICOS | TITULO-DO-CAPITULO-DO- | producoes.titulo<br>(tipo=CAPITULO)      |

Table 1: Mapeamento de campos XML do Lattes para o banco SPARK

**Atenção:** nos XMLs de exemplo analisados, o campo *DOI* em *ARTIGO-PUBLICADO* vem preenchido em ~88% dos artigos. O *ISSN* vem em 100% dos artigos. Para *TRABALHO-EM-EVENTOS*, o nó *DETALHAMENTO* não existe na maioria dos currículos — eventos não terão *ISSN* disponível e portanto não serão enriquecidos com *Qualis* nem *JCR*.

## 2.2 Planilha Qualis CAPES

A planilha Qualis é um arquivo estático (CSV/XLSX) obtido da Plataforma Sucupira. **Não existe API** — o arquivo precisa ser baixado manualmente e disponibilizado em diretório configurado no pipeline Hop.

### Como obter:

1. Acesse: <https://sucupira-legado.capes.gov.br/sucupira/public/consultas/coleta/veiculoPublicacaoQualis/listaConsultaGeralPeriodicos.jsf>
2. Em *Evento de Classificação* selecione: CLASSIFICAÇÕES DE PERIÓDICOS QUADRIÊNIO 2017-2020
3. Deixe os demais campos em branco e clique em *Exportar Excel*
4. Salve como `qualis_capes.csv` (converter de XLSX para CSV no Excel ou Libre-Office)

Colunas relevantes da planilha:

| Coluna            | Tipo               | Uso no pipeline                                    |
|-------------------|--------------------|----------------------------------------------------|
| ISSN              | string (8 dígitos) | Chave primária de lookup                           |
| Título            | string             | Lookup fallback (quando ISSN não localiza)         |
| Área de Avaliação | string             | Informativo (não armazenado no MVP)                |
| Estrato           | string (A1–C)      | <code>producoes.qualis</code> (máximo entre áreas) |

Table 2: Colunas relevantes da planilha Qualis CAPES

**Decisão de escopo:** *um mesmo periódico pode aparecer em múltiplas áreas com estratos diferentes. O pipeline guarda apenas o **estrato mais alto** encontrado ( $A1 > A2 > \dots > C$ ). Isso é consistente com o que sistemas similares como o SIMCC fazem. Ver Seção 5.1 para a implementação correta do MAX.*

### 2.3 CrossRef API

A CrossRef API é usada exclusivamente para completar o campo `doi` quando ele vem vazio no XML do Lattes. Não requer cadastro, mas adicionar o header `mailto` ativa o *Polite Pool* com limites mais generosos.

Endpoint utilizado:

```
GET https://api.crossref.org/works
?query.bibliographic={titulo_do_artigo}
&rows=1
&select=DOI,title,score
User-Agent: SPARK-ETL/1.0 (mailto:seu@email.uneb.br)
```

Lógica de uso:

- Se `producoes.doi` já vem preenchido no XML → **não chamar** a API
- Se `producoes.doi` está vazio → chamar com o título do artigo
- Se a resposta retornar `score > 70` → aceitar o DOI retornado
- Se `score ≤ 70` ou sem resultado → gravar NULL e registrar no log como `sem_match_doi`

Campos relevantes da resposta JSON:

| Campo JSON                  | Tipo   | Uso                                                |
|-----------------------------|--------|----------------------------------------------------|
| <code>items[0].DOI</code>   | string | <code>producoes.doi</code>                         |
| <code>items[0].score</code> | float  | Validação de confiança — aceitar somente se $> 70$ |

Table 3: Campos relevantes da resposta CrossRef

## 2.4 OpenAlex API

O OpenAlex é usado para buscar o Impact Factor (`2yr_mean_citedness`) de cada periódico via ISSN. É gratuito, mas requer registro de chave para 100.000 requisições/dia.

### Como obter a chave:

1. Acesse <https://openalex.org> e crie uma conta gratuita
2. Gere uma API key no dashboard
3. Configure no Apache Hop como variável de ambiente: `OPENALEX_API_KEY`

Endpoint utilizado:

```
GET https://api.openalex.org/sources/issn:{ISSN}
?select=issn_l,display_name,2yr_mean_citedness
&api_key={OPENALEX_API_KEY}
```

Campos relevantes da resposta:

| Campo JSON                      | Tipo         | Uso                                                          |
|---------------------------------|--------------|--------------------------------------------------------------|
| <code>2yr_mean_citedness</code> | float   null | <code>producoes.jcr</code> (substituto do Impact Factor JCR) |
| <code>display_name</code>       | string       | Validação cruzada com <code>nome_veiculo</code> do XML       |

Table 4: Campos relevantes da resposta OpenAlex

**Atenção:** o campo `2yr_mean_citedness` é equivalente ao JIF da Clarivate, mas não idêntico. A cobertura é de ~90% das fontes com ISSN. Periódicos não indexados retornam HTTP 404 — nesses casos gravar NULL em `producoes.jcr`.

## 3. Fase 1 — Extração dos XMLs

### 3.1 Configuração no Apache Hop

Usar o step **Get Files in Folder** para listar todos os arquivos `.xml` no diretório configurado, seguido de **Get Data From XML** para o parse de cada arquivo.

Configuração do step *Get Data From XML*:

- **Encoding:** ISO-8859-1 (padrão dos XMLs do Lattes)
- **Loop XPath:** `/CURRICULO-VITAE` (um registro por arquivo)
- **Campo extraído do nó raiz:** `NUMERO-IDENTIFICADOR` → `lattes_id`

Steps adicionais de *Get Data From XML* para sub-nós:

- XPath /CURRICULO-VITAE/DADOS-GERAIS → NOME-COMPLETO, ORCID-ID
- XPath /CURRICULO-VITAE/DADOS-GERAIS/RESUMO-CV → TEXTO-RESUMO-CV-RH
- XPath ../ARTIGO-PUBLICADO → loop por cada artigo
- XPath ../TRABALHO-EM-EVENTOS → loop por cada trabalho em evento
- XPath ../LIVRO-PUBLICADO-OU-ORGANIZADO → loop por cada livro
- XPath ../CAPITULO-DE-LIVRO-PUBLICADO → loop por cada capítulo

### 3.2 Mapeamento por Tipo de Produção

Cada tipo usa tags diferentes. O pipeline trata cada um como um fluxo separado e unifica ao final com **Append Streams**, discriminando pelo campo `tipo_producao`.

| Campo destino               | ARTIGO              | EVENTO          | LIVRO           | CAPÍTULO                    |
|-----------------------------|---------------------|-----------------|-----------------|-----------------------------|
| <code>tipo_producao</code>  | ARTIGO              | EVENTO          | LIVRO           | CAPITULO                    |
| <code>titulo</code>         | TITULO-DO-ARTIGO    | TITULO-DO-TRAB  | TITULO-DO-LIVRO | TITULO-DO-CAPITULO-DO-LIVRO |
| <code>ano_publicacao</code> | ANO-DO-ARTIGO       | ANO-DO-TRABALHO | ANO             | ANO                         |
| <code>doi</code>            | DOI                 | DOI             | DOI             | DOI                         |
| <code>nome_veiculo</code>   | TITULO-DO-PERIODICO | (sem campo)     | (título)        | TITULO-DO-LIVRO             |
| <code>issn</code>           | ISSN (det.)         | (sem campo)     | (sem campo)     | (sem campo)                 |

Table 5: Mapeamento de campos XML por tipo de produção

*TRABALHO-EM-EVENTOS não possui nó DETALHAMENTO na maioria dos currículos analisados. Isso significa que eventos não têm ISSN disponível e não serão enriquecidos com Qualis nem JCR. Os campos `qualis` e `jcr` ficam NULL para `tipo=EVENTO`.*

## 4. Fase 2 — Transformação e Normalização

### 4.1 Limpeza de ISSN

O ISSN nos XMLs do Lattes vem **sem hífen** (ex: 01234567). O OpenAlex e a planilha Qualis esperam o formato **com hífen** (0123-4567). O pipeline deve formatar o ISSN antes de qualquer lookup.

Regra de formatação (aplicar via step *Formula* ou *Modified JavaScript Value* no Hop):

1. Remover todos os caracteres não numéricos do campo ISSN bruto
2. Se resultado = 8 dígitos: inserir hífen na posição 4  
"01234567" -> "0123-4567"
3. Se comprimento != 8: gravar NULL + registrar no log

### 4.2 Normalização de Títulos

O título é usado como chave de busca no CrossRef quando o DOI está vazio. Aplicar no step *Modified JavaScript Value*:

- `trim()` — remover espaços no início e no fim
- Converter encoding de ISO-8859-1 para UTF-8
- Remover caracteres de controle e quebras de linha
- **Não** aplicar stemming nem remover stopwords — o CrossRef prefere o título completo

### 4.3 Deduplicação e UPSERT

Um mesmo pesquisador pode ter o currículo reprocessado em execuções futuras. O pipeline usa `INSERT ... ON CONFLICT DO UPDATE` baseado nas seguintes chaves:

| Tabela        | Chave de conflito                              | Comportamento                                    |
|---------------|------------------------------------------------|--------------------------------------------------|
| pesquisadores | lattes_id (UNIQUE)                             | Atualiza nome_completo, resumo, data_atualizacao |
| producoes     | pesquisador_id +<br>titulo +<br>ano_publicacao | Atualiza qualis, jcr, doi se vieram vazios antes |

Table 6: Chaves de conflito para UPSERT

## 5. Fase 3 — Enriquecimento

### 5.1 Lookup Qualis (CSV local)

O Apache Hop tem o step nativo **Stream Lookup** que faz join entre o fluxo de produções e um dataset em memória. A planilha Qualis é carregada uma única vez no início do pipeline.

#### Fluxo de lookup:

1. Carregar o CSV do Qualis em memória com o step *CSV File Input*
2. Formatar o ISSN da planilha no mesmo padrão com hífen
3. Criar coluna auxiliar `estrato_num`: A1=10, A2=9, A3=8, A4=7, B1=6, B2=5, B3=4, B4=3, B5=2, C=1
4. Usar *Stream Lookup* com join: `producoes.issn = qualis.ISSN`
5. Se houver múltiplos estratos para o mesmo ISSN (áreas diferentes), aplicar *Group By* com `MAX(estrato_num)`
6. Converter o número de volta para o estrato string

**Por que a coluna numérica?** A função `MAX` em string retorna resultado incorreto para os estratos: em ordem alfabética `A2 > A1`, mas `A1` é melhor. A coluna numérica auxiliar resolve isso sem precisar de lógica de comparação customizada.

**Fallback por nome do periódico:**

Se o join por ISSN não encontrar resultado (ISSN vazio ou não presente na planilha), tentar join pelo nome do periódico (Título da planilha Qualis vs `producoes.nome_veiculo`). Usar comparação case-insensitive com `trim()`. Registrar no campo `sem_match_qualis` do log os periódicos que não encontraram correspondência em nenhum dos dois métodos.

**5.2 Lookup DOI (CrossRef API)**

Executado apenas para registros onde `producoes.doi` está vazio após a extração do XML.

**Fluxo de chamada:**

1. Separar registros com DOI dos sem DOI via step *Filter Rows*
2. Para o fluxo sem DOI: usar step *HTTP Client* com a URL da CrossRef
3. Parsear JSON com step *JSON Input*; extrair `items[0].DOI` e `items[0].score`
4. Aplicar regra: `score > 70` → aceitar DOI; caso contrário → NULL
5. Reunificar os dois fluxos com *Append Streams*

Configuração do step *HTTP Client*:

```
URL      : https://api.crossref.org/works
Method   : GET
Params   :
  query.bibliographic = ${titulo_normalizado}
  rows                 = 1
  select               = DOI,title,score
Headers:
  User-Agent: SPARK-ETL/1.0 (mailto:${ETL_EMAIL})
```

**5.3 Lookup Impact Factor (OpenAlex API)**

Executado para todos os registros que possuem ISSN. Periódicos sem ISSN (eventos, livros) são ignorados nessa fase.

**Fluxo de chamada:**

1. Filtrar: apenas registros com ISSN preenchido
2. **Deduplicar ISSNs** — usar *Sort Rows* + *Unique Rows* por ISSN para não chamar a API duas vezes pelo mesmo periódico
3. Usar step *HTTP Client* com a URL do OpenAlex
4. Parsear JSON e extrair `2yr_mean_citedness`
5. Se HTTP 404 ou campo `null` → gravar NULL em `jcr`

6. Fazer join do resultado de volta ao fluxo principal pelo ISSN

Configuração do step *HTTP Client*:

```
URL      : https://api.openalex.org/sources/issn:${issn_formatado}
Method   : GET
Params   :
  select  = issn_l,display_name,2yr_mean_citedness
  api_key = ${OPENALEX_API_KEY}
```

**Otimização:** se o mesmo ISSN aparece em 10 artigos do mesmo pesquisador, a API só precisa ser chamada uma vez. O step *Unique Rows* por ISSN antes das chamadas HTTP evita requisições redundantes. A economia é significativa em pesquisadores prolíficos com muitos artigos no mesmo periódico.

## 6. Fase 4 — Carga no Supabase

### 6.1 Conexão JDBC

```
Host      : db.{PROJECT_REF}.supabase.co
Port      : 5432
Database  : postgres
User      : postgres
Password  : ${SUPABASE_DB_PASSWORD}
SSL       : require
```

Configurar as variáveis no ambiente de execução do Apache Hop (arquivo `.env` ou variáveis do sistema operacional). **Nunca** versionar credenciais no repositório Git.

### 6.2 Ordem de Carga

A ordem é obrigatória pelas foreign keys do schema:

1. Inserir/atualizar `pesquisadores` (sem dependências)
2. Inserir/atualizar `producoes` (FK `pesquisador_id` → `pesquisadores.id`)
3. Registrar na tabela `etl_logs`

### 6.3 UPSERT de Pesquisadores

```
INSERT INTO pesquisadores
  (lattes_id, nome_completo, resumo, data_atualizacao)
VALUES (?, ?, ?, NOW())
ON CONFLICT (lattes_id) DO UPDATE SET
  nome_completo      = EXCLUDED.nome_completo,
  resumo             = EXCLUDED.resumo,
  data_atualizacao   = NOW();
```

### 6.4 UPSERT de Produções

```
INSERT INTO producoes
```

```
(pesquisador_id, titulo, tipo_producao, ano_publicacao,
  nome_veiculo, issn, doi, qualis, jcr)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
ON CONFLICT (pesquisador_id, titulo, ano_publicacao) DO UPDATE SET
  doi          = COALESCE(EXCLUDED.doi,      producoes.doi),
  qualis       = COALESCE(EXCLUDED.qualis,   producoes.qualis),
  jcr          = COALESCE(EXCLUDED.jcr,      producoes.jcr),
  nome_veiculo = EXCLUDED.nome_veiculo;
```

*O uso de **COALESCE** garante que se a produção já existe com DOI preenchido e a nova carga traz **NULL**, o valor antigo é preservado. Isso evita perda de dados em reprocessamentos parciais.*

## 6.5 Registro de Log

```
INSERT INTO etl_logs
  (iniciado_em, finalizado_em, status,
   total_inseridos, total_atualizados, total_sem_match,
   detalhes)
VALUES (?, NOW(), ?, ?, ?, ?,
  '{
    "arquivos_processados": N,
    "sem_match_qualis": [...ISSNs sem correspondencia...],
    "sem_match_doi":     [...titulos sem DOI...],
    "sem_match_jcr":     [...ISSNs sem JCR...]
  }'::jsonb
);
```

## 7. Fase 5 — Acionamento do Worker de Embeddings

Após a carga no banco, o Apache Hop aciona o Worker via chamada HTTP ao endpoint interno do back-end FastAPI. O Worker identifica as produções sem embedding na tabela `vetores` e os gera usando o modelo local `all-MiniLM-L6-v2` (Sentence-Transformers), sem dependência de APIs pagas.

Chamada de acionamento (step *HTTP Client* no Hop):

```
POST https://{RAILWAY_URL}/internal/trigger-embeddings
Headers:
  Authorization: Bearer ${INTERNAL_API_KEY}
Body: {}
```

**O Worker executa internamente:**

1. `SELECT id, titulo FROM producoes WHERE id NOT IN (SELECT producao_id FROM vetores)`
2. Para cada produção: gerar embedding via Sentence-Transformers (`all-MiniLM-L6-v2`)
3. `INSERT INTO vetores (producao_id, embedding, modelo_llm)` para cada re-

sultado

O modelo *all-MiniLM-L6-v2* gera vetores de **384 dimensões**, compatível com o DDL (*VECTOR(384)*). O modelo é baixado automaticamente pelo *Sentence-Transformers* na primeira execução e não requer nenhuma API paga.

## 8. Mapa Completo de Correlação de Dados

A tabela abaixo resume todas as correlações entre fontes: qual campo é a chave de join, o que é buscado, onde é armazenado e o que acontece quando não há correspondência.

| Fonte                 | Chave de join            | Campo buscado | Campo destino | Sem correspondência                    |
|-----------------------|--------------------------|---------------|---------------|----------------------------------------|
| Qualis CSV            | issn<br>(formatado)      | estrato       | producoes.qu  | Tenta por nome; se falhar → NULL + log |
| Qualis CSV (fallback) | nome_veiculo             | estrato       | producoes.qu  | NULL + registra no log                 |
| CrossRef API          | titulo<br>(normalizado)  | DOI           | producoes.do  | NULL se score $\leq 70$ + log          |
| OpenAlex API          | issn<br>(formatado)      | 2yr_mean_cit  | producoes.jc  | NULL (404 ou campo ausente) + log      |
| XML Lattes (direto)   | DOI já preenchido no XML | —             | producoes.do  | N/A — vem direto do XML                |

Table 7: Mapa de correlação entre fontes de dados

## 9. Tratamento de Erros e Logs

### 9.1 Erros por arquivo XML

Se um arquivo XML estiver corrompido, com encoding inesperado ou com tag raiz inválida, o step de parse deve redirecionar o registro para uma stream de erro (*Error Handling* do Hop). O pipeline continua processando os demais arquivos e registra o arquivo com erro no campo `detalhes` do `etl_logs`.

### 9.2 Erros de API

CrossRef e OpenAlex podem retornar erros HTTP (429 rate limit, 5xx, timeout). Configurar retry com backoff no step *HTTP Client*:

- Máximo de 3 tentativas por requisição
- Espera entre tentativas: 2s, 4s, 8s (backoff exponencial)
- Se ainda assim falhar: gravar NULL no campo correspondente e continuar

### 9.3 Campos sem correspondência

O campo `detalhes` em `etl_logs` acumula três listas JSONB:

- **sem\_match\_qualis:** ISSNs não encontrados na planilha Qualis (nem por ISSN nem por nome)
- **sem\_match\_doi:** títulos de artigos sem DOI via CrossRef com score suficiente
- **sem\_match\_jcr:** ISSNs que retornaram 404 ou campo nulo no OpenAlex

Esses registros são visíveis para o Admin na aba ETL do painel do SPARK.

## 10. Checklist de Pré-Execução

---

Verificar antes de cada execução do pipeline:

1. Banco Supabase acessível e UNIQUE constraint de `producoes` adicionada
2. Arquivo `qualis_capes.csv` no diretório configurado e atualizado
3. Variáveis de ambiente configuradas:
  - `SUPABASE_DB_PASSWORD`
  - `OPENALEX_API_KEY`
  - `ETL_EMAIL`
  - `INTERNAL_API_KEY`
4. Diretório de XMLs populado com os currículos a processar
5. Worker de embeddings em execução no Railway
6. Espaço em disco suficiente para logs (~500 KB por execução com 50 currículos)

## 11. Referências

---

- CrossRef REST API: <https://api.crossref.org> — <https://github.com/CrossRef/rest-api-doc>
- OpenAlex API — Sources: <https://docs.openalex.org/api-entities/sources>
- Plataforma Sucupira — Qualis: <https://sucupira-legado.capes.gov.br/sucupira/public/consultas/coleta/veiculoPublicacaoQualis/listaConsultaGeralPeriodicos.jsf>
- Apache Hop — HTTP Client step: <https://hop.apache.org/manual/latest/pipeline/transforms/http.html>
- Supabase — pgvector: <https://supabase.com/docs/guides/database/extensions/pgvector>
- Sentence-Transformers — all-MiniLM-L6-v2: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>